

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumnos

Agustín Emiliano Sánchez

Maico Agustín Vivas

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Organización Empresarial

Docentes Titulares

Ariel Enferrel

Martin A. García

Cinthia Rigoni

15 de junio de 2026

INDICE

Introducción y objetivo	3
Marco teórico	4-5
Arquitectura y estructura de datos	6
Diagrama de flujo	7
Funcionalidades del sistema	8-9
Dificultades	10
Conclusión	11
Bibliografía	12

Introducción

El presente Trabajo Práctico Integrador (TPI) corresponde a la materia Programación I de la Tecnicatura Universitaria en Programación a Distancia de la UTN. Su propósito es consolidar los conocimientos adquiridos durante la cursada mediante el desarrollo de una aplicación real en Python que gestiona información sobre países.

El sistema permite cargar un dataset desde un archivo CSV y ofrece al usuario un menú interactivo en consola para realizar búsquedas, filtros, ordenamientos y cálculos estadísticos sobre los datos. A través de esta experiencia, se ponen en práctica conceptos fundamentales como listas, diccionarios, funciones, condicionales, estructuras repetitivas y manejo de archivos.

Objetivo General

Desarrollar una aplicación en Python que permita gestionar un dataset de países, aplicando estructuras de datos, modularización con funciones y técnicas de filtrado y ordenamiento.

Objetivos Específicos

- Implementar la lectura de datos desde un archivo CSV utilizando el módulo estándar de Python.
- Almacenar la información de países usando listas de diccionarios.
- Desarrollar funciones modulares con responsabilidad única para cada operación.
- Implementar búsquedas por nombre con coincidencia parcial o exacta.
- Aplicar filtros por continente, rango de población y rango de superficie.
- Ordenar el dataset por nombre, población o superficie en ambas direcciones.
- Calcular y mostrar estadísticas generales del conjunto de datos.
- Validar entradas y manejar errores de forma robusta y clara.

Marco Teórico

A continuación, se describen los conceptos fundamentales aplicados en el desarrollo del sistema, fundamentados en los contenidos de la cátedra.

Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar colecciones de elementos de cualquier tipo. Se definen con corchetes y admiten operaciones de inserción, eliminación, búsqueda y recorrido. En este proyecto, la lista es la estructura principal que contiene todos los registros de países cargados desde el CSV.

Ejemplo de uso en el sistema:

```
países = [] → lista principal del sistema
```

Diccionarios

Los diccionarios permiten almacenar datos mediante pares clave-valor, lo que facilita el acceso a la información. Cada país se representa como un diccionario con cuatro campos: nombre, población, superficie y continente. Esta elección mejora la legibilidad del código respecto al uso de listas simples o tuplas.

```
país = {"nombre": "Argentina", "población": 45376763, "superficie": 2780400, "continente": "América"}
```

Funciones

Las funciones son bloques de código reutilizables que se definen con la palabra reservada `def`. Cada función debe encargarse de una sola tarea, lo que facilita el mantenimiento, la prueba y la comprensión del código. El sistema respeta el siguiente orden: hay funciones separadas para agregar, buscar, filtrar, ordenar, actualizar y mostrar estadísticas.

Estructuras Condicionales y Repetitivas

Las estructuras condicionales (`if / elif / else`) permiten dirigir el flujo del programa según condiciones evaluadas en tiempo de ejecución. Se usan en cada validación de entrada y en la lógica del menú principal.

Las estructuras repetitivas (`for, while`) permiten recorrer listas y repetir procesos. El recorrido del dataset para aplicar filtros y cálculos estadísticos se realiza con bucles `for`.

Archivos CSV

Un archivo CSV (Comma-Separated Values) es un formato de texto plano para almacenar datos tabulares. Python provee el módulo `csv` de su biblioteca estándar, que ofrece las clases `csv.reader` y `csv.DictReader` para leer archivos con o sin encabezado, respectivamente. En este proyecto se utiliza `csv.DictReader` para mapear automáticamente cada fila a un diccionario con las claves del encabezado.

Ordenamientos

Python provee la función `sorted()` y el método `list.sort()`, ambos con soporte para una función `key` que permite ordenar objetos complejos por un atributo específico. El parámetro `reverse=True` habilita el orden descendente. Se utiliza `sorted()` para no modificar la lista original y trabajar sobre una copia ordenada.

Estadísticas Básicas

Las estadísticas calculadas hacen uso de las funciones integradas `max()`, `min()`, `sum()` y `len()` de Python. Combinadas con expresiones generadoras y la función `key` de `max/min`, permiten obtener el país con mayor y menor población, el promedio de población y superficie, y la cantidad de países por continente de forma eficiente sin librerías externas.

Arquitectura del Sistema

El sistema adopta una arquitectura modular basada en funciones. Cada módulo lógico agrupa responsabilidades relacionadas, favoreciendo la legibilidad, el mantenimiento y la escalabilidad.

```
tpi-programacion/  
├─ main.py           # Punto de entrada y menú principal  
├─ datos/  
│   └─ paises.csv    # Dataset base  
├─ modulos/  
│   ├── archivos.py  # Cargar y guardar el CSV  
│   ├── paises.py     # Mostrar, agregar, actualizar y buscar países  
│   ├── filtros.py    # Filtros por continente, población y superficie  
│   ├── ordenamientos.py # Ordenamiento por distintos criterios  
│   ├── estadisticas.py # Cálculo y muestra de estadísticas  
│   └─ normalizar.py  # Normalización de texto (minúsculas y sin acentos)  
└─ README.md
```

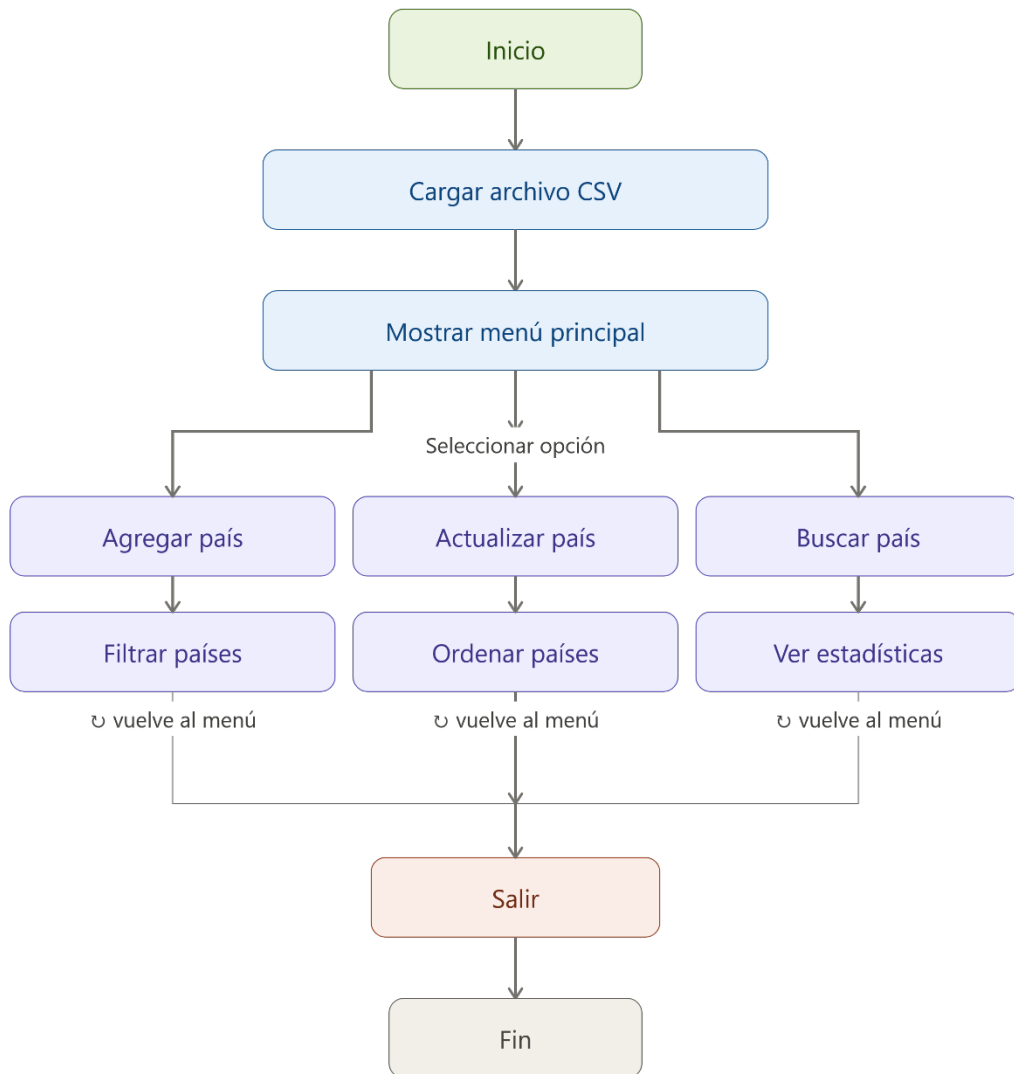
Estructura de Datos Principal

Cada país se representa como un diccionario dentro de una lista global:

```
países = [ {"nombre": "Argentina", "población": 45376763, "superficie": 2780400,  
            "continente": "América"}, {"nombre": "Japón", "población": 125800000, "superficie": 377975,  
            "continente": "Asia"} ]
```

Diagrama de Flujo

El siguiente diagrama describe el flujo general de operaciones del sistema:



Luego de cada operación (excepto Salir), el sistema vuelve automáticamente al menú principal.

Funcionalidades del Sistema

Carga desde CSV

Al iniciar, el programa lee el archivo `países.csv` desde el directorio del proyecto.

Agregar País

El sistema solicita los cuatro campos obligatorios: nombre, población, superficie y continente. Ninguno puede quedar vacío. Los valores numéricos son validados para asegurar que sean enteros positivos. Si ya existe un país con el mismo nombre (comparación sin distinguir mayúsculas), el sistema alerta al usuario y cancela la operación.

Actualizar País

Se busca el país por nombre exacto. Si se encuentra, se permite modificar los valores de población y superficie.

Buscar País

La búsqueda es insensible a mayúsculas y admite coincidencias parciales. Por ejemplo, buscar “arg” retorna “Argentina”. Si no se encuentran resultados, se muestra un mensaje indicando que no hay coincidencias.

Filtros

1. Por continente: muestra todos los países del continente ingresado (sin distinguir mayúsculas).
2. Por rango de población: solicita un mínimo y un máximo e incluye los límites.
3. Por rango de superficie: igual que el anterior, pero sobre el campo superficie (km²).

Ordenamientos

El sistema permite ordenar el dataset completo por tres criterios, en ambas direcciones:

- Nombre (alfabético ascendente / descendente)
- Población (ascendente / descendente)
- Superficie (ascendente / descendente)

El ordenamiento se aplica sobre una copia del dataset para no alterar el orden original.

Estadísticas

El sistema calcula y presenta los siguientes indicadores:

- País con mayor población
- País con menor población
- Promedio de población
- Promedio de superficie
- Cantidad de países por continente
- Salir de estadísticas

Dificultades

Durante el desarrollo surgieron algunos inconvenientes típicos de trabajar con datos cargados desde un archivo externo.

Uno de los primeros problemas fue el manejo de valores faltantes o mal escritos en el CSV: al intentar convertir ciertos campos a números, el programa se interrumpía con errores. Esto se resolvió incorporando bloques try/except en todas las conversiones de tipo, lo que permitió que el sistema avisara al usuario sin detenerse.

Otra dificultad fue lograr que las búsquedas y comparaciones de nombres de países funcionaran sin importar cómo el usuario escribiera el texto (mayúsculas, minúsculas o combinaciones). Se solucionó normalizando todo con `.lower()` antes de comparar, lo que evitó resultados inconsistentes.

También costó un poco decidir cómo ordenar los datos sin perder la información original. Probandos con `list.sort()` notamos que modificaba la lista original, lo cual complicaba mostrar distintos órdenes sin afectar el resto del programa. Cambiar a `sorted()` resolvió esto de forma simple, ya que genera una copia ordenada sin tocar la base de datos en memoria.

Este trabajo nos permitió entender mejor por qué ciertas herramientas de Python (como `csv.DictReader`, `sorted()` o el manejo de excepciones) existen y qué problemas concretos resuelven, más allá de la teoría.

También reforzamos la importancia de probar el programa con datos "raros" (vacíos, mal escritos, en mayúsculas) para detectar errores antes de que los encuentre el usuario final. En general, el proyecto ayudó a consolidar conceptos de estructuras de datos, manejo de archivos y control de errores de manera práctica.

Conclusión

Hacer este Trabajo Práctico Integrador nos permitió poner en práctica, de forma conjunta, todo lo que fuimos viendo durante la cursada de Programación I. Armar una aplicación real, con lectura y escritura de archivos, validaciones y varias funcionalidades distintas, fue una experiencia que nos ayudó mucho a ordenar la forma de pensar los problemas y a escribir código más prolijo y organizado.

Algo que notamos que marcó una gran diferencia fue dividir el programa en funciones, cada una encargada de una sola tarea. Esto no solo hizo que el código fuera más fácil de leer, sino que también nos facilitó muchísimo encontrar y corregir errores, ya que sabíamos exactamente dónde buscar cuando algo no funcionaba como esperábamos. Del mismo modo, usar diccionarios para representar cada país terminó siendo una decisión muy acertada, porque simplificó bastante la forma de acceder y manipular los datos.

Respecto al trabajo en equipo, dividir las tareas por módulos nos permitió avanzar en paralelo sin pisarnos entre nosotros, y después integrar todo fue relativamente sencillo. Mantener una comunicación constante y revisar el código entre todos también ayudó bastante a mejorar la calidad final del trabajo y a detectar cosas que, individualmente, quizás se nos hubieran pasado por alto.

Bibliografía

1. Material de la cátedra — Programación I, UTN.
2. Inteligencia Artificial (Claude y ChatGPT).
3. CSV - <https://docs.python.org/3/library/csv.html>
4. Unicodedata - <https://docs.python.org/3/library/unicodedata.html>
5. YouTube <https://www.youtube.com/maxiprograma>
6. YouTube <https://www.youtube.com/c/charlycimino>

GitHub: <https://github.com/agussanchz/tpi-programacion>

Video explicativo: <https://youtu.be/UbDF35t3I7k>